NASA KOI Exoplanet Classification

Random Forest with a Train / Validation / Test Protocol

Project report and viva preparation guide

Item	Value
Task	Binary classification: CONFIRMED exoplanet vs FALSE POSITIVE
Data	NASA Exoplanet Archive, Kepler Objects of Interest (KOI) cumulative table, 9,564 rows
Model	Random Forest (165 trees, max depth 30)
Data split	70% train (4,637) / 15% validation (994) / 15% test (994), stratified
Final test result	Accuracy 90.54%, Recall 93.40%, ROC-AUC 0.9731
Interfaces	Tkinter GUI (gui.py) and command line (main.py)

1. What the project does

NASA's Kepler telescope watched about 150,000 stars for the tiny dips in brightness that happen when a planet passes in front of its star (a **transit**). Each detection is called a Kepler Object of Interest (KOI). After follow-up, each KOI is labelled **CONFIRMED** (a real planet), **FALSE POSITIVE** (something else, such as an eclipsing binary star or instrument noise) or **CANDIDATE** (undecided).

Our program learns, from measurable properties of the signal and the star, to separate real planets from false positives. This is **supervised binary classification**: we give the model examples with known answers, and it learns a rule it can apply to new objects.

2. Dataset and features

The raw file has 141 columns. We use nine features and the label `koi_disposition`.

Feature	Meaning	Why it helps
koi_depth	How much the star dims during transit (ppm)	Deep dips can mean a large body, often a binary star
koi_duration	Length of the transit (hours)	Must be physically consistent with the star and orbit
koi_impact	How centrally the planet crosses the star disc	Grazing crossings are more often false alarms
koi_model_snr	Signal-to-noise ratio of the transit fit	Weak signals are less trustworthy
koi_max_mult_ev	Strongest multiple-event statistic	Measures how consistently the signal repeats
koi_srad / koi_smass / koi_steff	Star radius, mass, temperature	Give physical context to judge the signal
koi_bin_oedp_sig	Odd-even depth difference significance	Odd and even transits differing suggests an eclipsing binary

3. Preprocessing methodology (and why)

Raw data cannot go straight into a model. Every step below is logged with row counts in `Performance/<model>_preprocessing.json`.

Step	What we do	Why	Rows after
1. Load	Read CSV, skip lines starting with #	NASA files start with a metadata block that is not data	9,564
2. Select features	Keep 9 features + target	Out of 141 columns these are the attributes the project uses	9,564
3. Remove CANDIDATE	Drop 1,977 rows	Candidates have no confirmed label, so they cannot teach or test a supervised model; this makes the task binary	7,587
4. Remove missing values	Drop 962 rows with any missing feature	scikit-learn's Random Forest cannot handle missing values; deleting avoids inventing data	6,625
5. Encode label	FALSE POSITIVE = 0, CONFIRMED = 1	Models and metrics need numbers	6,625
(No scaling)	Deliberately skipped	Trees split on the ordering of values, so scaling does not change a Random Forest	-

Final class balance: 3,895 FALSE POSITIVE (58.8%) and 2,730 CONFIRMED (41.2%). Mildly imbalanced, which is why we report balanced accuracy, MCC and ROC-AUC besides plain accuracy.

Trade-off of deleting rows: we lose 962 rows and, if missingness is not random, may bias the data a little. Imputation (for example the median) is an alternative, but it must be fitted on the training set only, otherwise it leaks information. We kept deletion because it is simple and matches the original project.

4. Train / validation / test split

The problem we fixed. The original code trained on all rows and then evaluated on those same rows (through cross-validation), and the default threshold 0.46 had been chosen by looking at the same data. The reported scores were therefore optimistic. The honest question is: *how well does the model do on data it never saw and never influenced any decision?* To answer it, data is divided by purpose.

Set	Share	Rows	CONFIRMED share	Purpose
Training	70%	4,637	41.2%	Fit the model; run 10-fold cross-validation
Validation	15%	994	41.3%	Choose the decision threshold. Never used for fitting
Test	15%	994	41.1%	Final, one-time, unbiased evaluation. Nothing is tuned on it

- **Stratified:** each set keeps the same class ratio (about 41% CONFIRMED), so no set is accidentally easier or harder.
- **random_state = 42:** the split is identical every run, so results are reproducible.
- **Why 70/15/15?** A common convention. With about 1,000 rows in test, the standard error of accuracy is about sqrt(0.9 x 0.1 / 994) = 0.95 percentage points, so a test score is accurate to roughly plus or minus 2 points at 95% confidence. Smaller test sets would be noisier; a smaller training set would give a weaker model.
- **Where to see the split data:** `Datasets/Splits/<model>_train.csv`, `_validation.csv`, `_test.csv`. They contain the exact rows used.
- **Rule:** if you look at test results and then change settings to improve them, the test set is no longer unbiased. Treat it as a final exam.

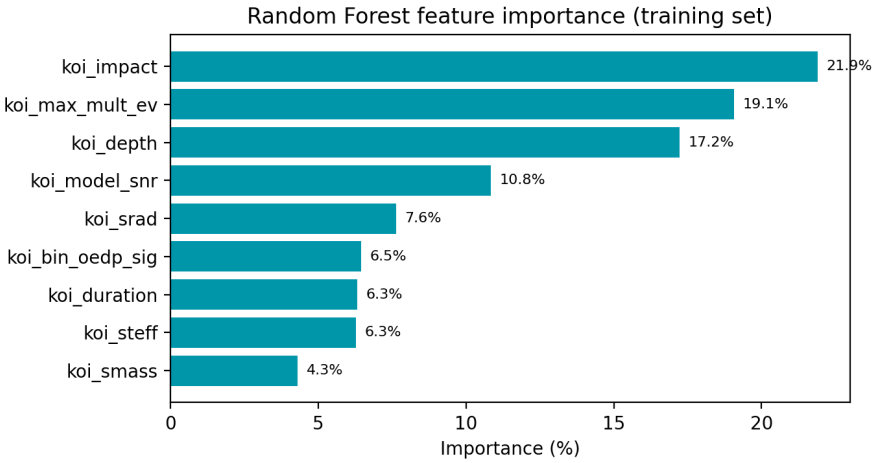
5. The model: Random Forest

A **decision tree** asks a chain of yes/no questions about the features (for example 'is koi_model_snr above 12?') until it reaches a verdict. One tree overfits easily. A **Random Forest** trains many trees, each on a random bootstrap sample of the rows and a random subset of features at each split, then averages their votes. The averaging cancels out individual trees' mistakes (lower variance), and the output *probability* is the fraction of trees voting CONFIRMED.

Hyperparameters we use

Parameter	Value	Meaning
n_estimators	165	Number of trees. More trees = more stable, with diminishing returns
max_depth	30	Longest allowed chain of questions in a tree. Limits overfitting
random_state	42	Fixes randomness for reproducibility
Everything else	scikit-learn defaults	criterion = gini, max_features = sqrt(9) = 3, bootstrap = True, min_samples_split = 2

Which features matter most?



Impact parameter, the multiple-event statistic and transit depth carry the most weight, which is physically sensible: they describe the shape and consistency of the dip. (Importance values come from a model fitted on the training set.)

6. Why Random Forest and not another model?

We kept Random Forest because it was the model in your original project, and then checked it against six alternatives on the **same training data**, using 10-fold stratified cross-validation inside the training set only (so the test set stays untouched). Other models used default settings; those needing scaled inputs got a StandardScaler inside the pipeline.

Model	CV accuracy	CV F1	CV ROC-AUC	AUC std
Random Forest	0.9019	0.8828	0.9604	0.0061
Gradient Boosting	0.8933	0.8755	0.9590	0.0067
Decision Tree	0.8449	0.8107	0.8392	0.0133
Logistic Regression	0.7434	0.6950	0.8313	0.0143
SVM (RBF)	0.8010	0.7822	0.8812	0.0086
KNN (k=5)	0.7966	0.7713	0.8587	0.0082
Naive Bayes	0.6237	0.6813	0.7979	0.0123

Reading the table. Random Forest and Gradient Boosting are statistically tied (AUC 0.9604 vs 0.9590, a gap far smaller than the fold-to-fold standard deviation of about 0.006). Every other model is clearly worse.

- **Decision Tree:** a single tree overfits; the forest fixes exactly that (AUC 0.839 vs 0.960).
- **Logistic Regression:** draws one straight boundary, but the planet/false-positive boundary is non-linear (AUC 0.831).
- **SVM and KNN:** can model non-linear boundaries but are sensitive to feature scaling and outliers, and slower at prediction (KNN) or tuning (SVM). AUC 0.881 and 0.859.
- **Naive Bayes:** assumes features are independent, which is false here (stellar radius, mass and temperature are strongly related). AUC 0.798.
- **Gradient Boosting:** a serious competitor. It builds trees sequentially, each correcting the last. It is more sensitive to hyperparameters and easier to overfit; in our CV it was equal to or marginally below Random Forest.

Honest note for the viva. In a quick swap test, Gradient Boosting scored higher than Random Forest on the test set (92.05% vs 90.54% accuracy). We deliberately did **not** switch models because of that: choosing a model by its test score contaminates the test set. The fair comparison is the cross-validation above (tied), so Random Forest remains justified. If you want to adopt Gradient Boosting, justify it with validation or CV results, not the test number.

Why Random Forest suits this data

- Works well on small to medium tabular data with mixed-scale features and no scaling needed.
- Captures non-linear relationships and feature interactions automatically.
- Robust to outliers (transit depth has extreme values) and hard to overfit badly.
- Gives probabilities, which we need for threshold tuning and ROC-AUC, and gives feature importance for interpretation.
- Fast to train (about 2 seconds here).

7. How to change the model

All model construction is in one function in `Code/modelTraining.py`, called `create_random_forest()`. Change what it returns and everything else (splitting, cross-validation, threshold tuning, metrics, graphs) works unchanged. The model only needs a `predict_proba` method.

Example A: Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

def create_random_forest():
    return GradientBoostingClassifier(n_estimators=200,
                                      learning_rate=0.1,
                                      random_state=42)
```

Example B: SVM (needs scaling, so wrap in a pipeline)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

def create_random_forest():
    return make_pipeline(StandardScaler(),
                         SVC(probability=True, random_state=42))
```

Notes: (1) The scaler lives inside the pipeline, so it is fitted on training data only in each fold, which avoids leakage. (2) The labels 'Random Forest' and 'RF' inside the performance JSON and the graph titles are text strings, so edit them to match the new model. (3) The JSON writer uses `default=str`, so models with non-standard parameter objects (such as pipelines) save correctly. We tested both examples: they run end to end.

To change Random Forest settings instead, edit the arguments, for example `n_estimators=300, max_depth=None`. Compare changes using validation or cross-validation scores, never the test score.

8. The decision threshold

A Random Forest outputs a probability. The **threshold** turns it into a verdict: probability >= threshold means 'exoplanet'. The default 0.5 is not always best.

How we choose it. We scan every possible threshold on the **validation** set and keep the one with the highest F1. Our run chose **0.382**. This is below 0.5, so the model is slightly more willing to say 'exoplanet', which raises recall (we miss fewer real planets) at the cost of precision (a few more false alarms). In

astronomy that is a sensible bias: a false alarm costs a follow-up check, a missed planet may never be re-examined.

You can override it in the GUI (untick 'Tune threshold automatically'). The old default 0.46 can still be entered, but it was chosen using the full dataset and is not validated.

9. Cross-validation

10-fold stratified cross-validation is run only on the training set. The training data is cut into 10 parts; the model trains on 9 and is scored on the remaining one, 10 times. We report the mean and standard deviation of each metric.

- **Why:** it shows how *stable* the model is. A small standard deviation (ours is about 1 point for accuracy) means the score does not depend on a lucky split.
- **Why not use it as the final score?** Cross-validation scores can still be nudged by repeated tuning. The untouched test set is the final judge.
- It is run in parallel (joblib, all CPU cores) and takes about 18 seconds.

Metric	CV mean	CV std
accuracy	0.8952	0.0100
precision	0.8273	0.0163
recall	0.9430	0.0140
f1	0.8812	0.0105
roc_auc	0.9604	0.0064

10. Performance metrics explained

Let TP = real planets found, TN = false positives correctly rejected, FP = false positives wrongly called planets, FN = real planets missed.

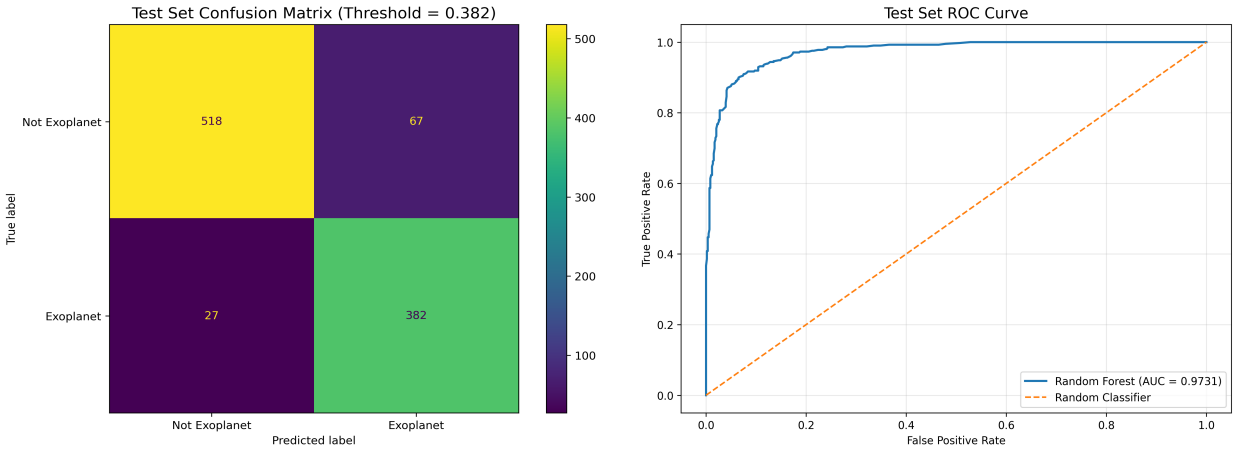
Metric	Formula	Plain meaning
Accuracy	(TP+TN) / all	Share of all predictions that were right
Balanced accuracy	(Recall + Specificity) / 2	Accuracy that is not dominated by the larger class
Precision	TP / (TP+FP)	Of objects called planets, how many really are
Recall (sensitivity)	TP / (TP+FN)	Of real planets, how many we found
Specificity	TN / (TN+FP)	Of false positives, how many we correctly rejected
F1	2PR / (P+R)	Single score balancing precision and recall
MCC	correlation of prediction and truth	One robust number for imbalanced data: 1 = perfect, 0 = random
ROC-AUC	area under ROC curve	How well the model ranks planets above false positives at every threshold. 0.5 = chance, 1 = perfect. Independent of the chosen threshold

11. Our results and what they mean

Metric	Validation	TEST
accuracy	0.8944	0.9054
balanced_accuracy	0.9003	0.9097
precision	0.8308	0.8508
recall	0.9341	0.9340
specificity	0.8664	0.8855
f1	0.8794	0.8904
mcc	0.7903	0.8103
roc_auc	0.9576	0.9731

Test confusion matrix: TN = 518, FP = 67, FN = 27, TP = 382 (994 test objects).

- **Accuracy 90.5%:** about 9 in 10 test objects classified correctly.
- **Recall 93.4%:** of the 409 real planets in the test set, we found 382 and missed 27.
- **Precision 85.1%:** of the 449 objects we called planets, 382 were real and 67 were false alarms.
- **Specificity 88.5%:** 518 of 585 false positives correctly rejected.
- **ROC-AUC 0.973:** if you pick one real planet and one false positive at random, the model gives the planet the higher probability about 97% of the time.
- **Validation vs test are close** (accuracy 89.4% vs 90.5%), and both match the cross-validation (89.5%). That consistency indicates the model is not overfitted to any one split. The test being slightly higher is normal sampling noise (about plus or minus 1 point).
- **These numbers are likely lower than the old code's**, which is expected and correct: the old scores were optimistic because of data leakage.



12. Where every output is saved

All paths are relative to the project root (the folder that contains Code/).

Output	Location	Contents
Cleaned dataset	Datasets/Processed.csv	Data after preprocessing, 6,625 rows
Training set	Datasets/Splits/<model>_train.csv	4,637 rows
Validation set	Datasets/Splits/<model>_validation.csv	994 rows

Output	Location	Contents
Test set	Datasets/Splits/<model>_test.csv	994 rows
Trained model	Models/<model>.pkl	Saved with joblib; reload with joblib.load
Preprocessing log	Performance/<model>_preprocessing.json	Rows removed at each step
Performance report	Performance/<model>_perf.json	Hyperparameters, split sizes, threshold, CV, validation and test metrics, timings
Test predictions	Performance/<model>_test_predictions.csv	Per test row: true label, predicted label, probability, correct or not, plus features
Test report	Performance/<model>_test_report.txt	Classification report and confusion matrix
Graphs	Graphs/Final Evaluation/test_roc_curve.png, test_confusion_matrix.png, test_metrics.png	Test-set visuals (300 dpi)

13. Code structure

File	Role
Code/preProcess.py	preProcess(): cleaning with logging. split_dataset(): stratified 70/15/15 split
Code/modelTraining.py	Model factory, metrics, threshold tuning, cross-validation, test evaluation, graphs, saving
Code/pipeline.py	run_pipeline(): preprocess then train/validate/test. Shared by both interfaces so they behave identically
Code/main.py	Command-line interface (asks 4 questions)
Code/gui.py	Tkinter window. Training runs in a background thread and sends messages through a queue, so the window never freezes

Data flow

```
RawData.csv -> preProcess -> Processed.csv -> split_dataset -> train / validation / test
   train      -> fit final model  +  10-fold CV
   validation -> choose threshold (max F1)
   test       -> final metrics, predictions CSV, graphs
   everything -> Models/*.pkl and Performance/*.json
```

14. How to run

- Install: `pip install pandas scikit-learn matplotlib joblib` (Tkinter ships with Python on Windows/macOS).
- Folder layout: Code/ must sit beside Datasets/, Models/, Performance/, Graphs/.
- `cd Code` then `py gui.py` (GUI) or `py main.py` (command line).
- GUI: pick the input CSV, keep 'Preprocess raw data first' ticked for raw NASA data, enter a model name, keep threshold tuning on, click Run.
- Windows note: if 'An Application Control policy has blocked this file' appears, Smart App Control is blocking scipy/scikit-learn DLLs; turn it off in Windows Security > App and browser control, or run in Google Colab.

15. Limitations and future work

- **Hyperparameters** (165 trees, depth 30) were inherited. If they were chosen using the full dataset, they carry slight leakage. Re-tune with GridSearchCV or RandomizedSearchCV on the training set only.

- **Row deletion** removed 962 objects. Try imputation (fitted on training data only) and compare.
- **Single hold-out test set** of 994 rows has about plus or minus 2 points of uncertainty. Repeating with different random seeds would quantify it.
- **Threshold** maximises F1; a different cost balance (for example prioritising recall) would choose another value.
- **Model choice:** Random Forest and Gradient Boosting are tied in CV; a tuned Gradient Boosting or an ensemble could be explored.
- **Scope:** uses only 9 of 141 columns. Adding features, or using the CANDIDATE objects as a prediction target after training, are natural extensions.

16. Viva questions and model answers

Q1. What is the problem you solve?

Classifying Kepler objects of interest as confirmed exoplanets or false positives from nine measured features. It is supervised binary classification.

Q2. Why did you remove CANDIDATE rows?

They have no ground-truth label, so a supervised model can neither learn from nor be scored on them.

Q3. Why did you delete rows with missing values?

scikit-learn's Random Forest cannot take missing values. Deletion is simple and does not invent data. Imputation is an alternative but must be fitted on the training set only.

Q4. Why no feature scaling?

Random Forests split on the ordering of values, so scaling does not change them. Scale-sensitive models (SVM, KNN, logistic regression) would need it.

Q5. How did you split the data and why?

Stratified 70/15/15 into train (4,637), validation (994) and test (994). Train fits the model, validation chooses the threshold, test gives one unbiased final score. Separating them prevents data leakage.

Q6. What was wrong with the original code?

It trained and evaluated on the same data and chose the threshold on it too, so scores were optimistic.

Q7. What is stratification?

Keeping the class ratio (about 41% CONFIRMED) the same in every set.

Q8. Why is the validation set separate from the test set?

The threshold is a tuning decision. If we tuned it on the test set, the test score would no longer be an honest estimate of unseen performance.

Q9. What is cross-validation and why did you use it?

Training on 9 folds and testing on the 10th, repeated 10 times. It measures stability. We run it on the training set only.

Q10. Explain how a Random Forest works.

Many decision trees, each trained on a bootstrap sample and a random subset of features per split, vote. Averaging reduces variance and overfitting. The probability is the fraction of trees voting CONFIRMED.

Q11. Why Random Forest and not other models?

It handles non-linear, mixed-scale tabular data without scaling, resists outliers, gives probabilities and feature importance, and trains fast. In 10-fold CV on the training set it scored AUC 0.960, tied with Gradient Boosting and clearly ahead of SVM (0.881), KNN (0.859), decision tree (0.839), logistic regression (0.831) and naive Bayes (0.798).

Q12. Gradient Boosting scored higher on test. Why not use it?

Selecting a model by test score contaminates the test set. In CV the two are tied within noise. We would only switch on the basis of validation or CV evidence.

Q13. How do you change the model?

Edit create_random_forest() in modelTraining.py to return another scikit-learn classifier with predict_proba, wrapping scale-sensitive models in a pipeline with StandardScaler. Everything else works unchanged.

Q14. What does the threshold do and how did you pick it?

It converts probability to a yes/no verdict. We scanned all thresholds on the validation set and picked the max-F1 one (0.382). It favours recall.

Q15. Define precision and recall in this context.

Precision: of objects we call planets, the fraction that are real (85.1%). Recall: of real planets, the fraction we find (93.4%).

Q16. What is ROC-AUC?

The probability that the model ranks a random real planet above a random false positive. 0.973 means excellent separation; 0.5 is chance. It is independent of the threshold.

Q17. Why report MCC and balanced accuracy?

The classes are imbalanced (59/41). These are robust to that, whereas accuracy alone can mislead.

Q18. Is the model overfitting?

Unlikely: CV (89.5%), validation (89.4%) and test (90.5%) accuracy are consistent, and CV standard deviation is only about 1 point.

Q19. What are the biggest FP and FN counts?

On test: 67 false alarms and 27 missed planets. The threshold trades these against each other.

Q20. Where are the split datasets and outputs?

Datasets/Splits for train/validation/test CSVs; Performance for JSON reports and test predictions; Graphs/Final Evaluation for plots; Models for the .pkl.

Q21. What are the limitations?

Inherited hyperparameters, row deletion, a single hold-out test set (about plus or minus 2 points), and a threshold optimised for F1 only.

Q22. How would you improve it?

Tune hyperparameters on the training set only, try imputation, repeat the split with several seeds, compare tuned Gradient Boosting, and explore cost-sensitive thresholds.